CARTESIANA — Gemelo digital con conexión TCP/IP

Seminario de Informática I · Trabajo final: Gemelos digitales · Documento de virtualización

Este documento describe solo lo que agrega la virtualización: la arquitectura de comunicación, el controlador de Webots que hace de PC industrial, el ciclo de inicialización, el protocolo de texto y el programa externo que registra y ejecuta trayectorias. El modelo mecánico (ejes, medidas, cinemática) está en el README del proyecto y no se repite acá.

1. Arquitectura

CARTESIANA tiene a bordo una **PC industrial que escucha por TCP/IP**. En el gemelo digital, ese papel lo cumple el controlador de Webots: es el **servidor TCP**. El programa externo es el **cliente** y se conecta directamente, sin puertos serie ni programas intermedios.

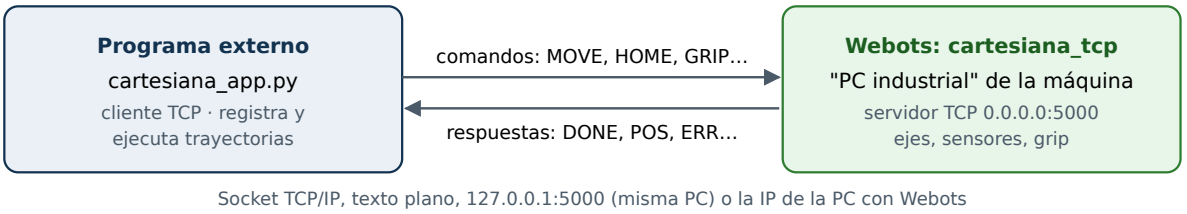


Figura 1. Cadena de comunicación: un socket TCP con líneas de texto en los dos sentidos.

No es una API REST. No hay URLs ni pedidos HTTP: el cliente abre **una** conexión TCP que queda abierta y por la que viajan líneas de texto, un comando y su respuesta por vez. Si Webots corre en otra PC, se usa su IP y se abre el puerto 5000 en el firewall.

2. Programas y archivos nuevos

Archivo	Función
`worlds/cartesiana_tcp.wbt`	Mundo del gemelo digital: robot con grip y sensores de inicio de carrera, dos zócalos de ejemplo y una pieza.
`controllers/cartesiana_tcp/cartesiana_tcp.py`	PC industrial simulada: servidor TCP no bloqueante, cola de comandos, ciclo de inicialización, ejes, grip y traslado de la pieza.
`controllers/cartesiana_tcp/protocol.py`	Análisis y formato de los mensajes. No depende de Webots.
`config/comunicacion.json`	Host y puerto del servidor.
`programa_externo/cartesiana_app.py`, `enlace.py`	Programa externo (Python + Tkinter, solo biblioteca estándar). `enlace.py` es el cliente TCP.
`programa_externo/trayectorias/*.json`	Trayectorias de ejemplo: `pick_and_place_A_a_B` y `ciclo_A_B_A`.
`tools/build_model.py` (modificado)	Genera el grip, los sensores, los zócalos y el mundo a partir de `config/robot_parameters.json`.

3. Ciclo de inicialización (homing)

La máquina real tiene **encoders relativos**: al encenderse no sabe dónde está. Cuenta pasos desde donde quedó, pero esa posición es desconocida. Por eso cada eje tiene un **sensor de inicio de carrera** y, antes de trabajar, la máquina hace un ciclo de inicialización. La simulación reproduce ese comportamiento, aunque Webots sí conoce la posición real.

1. **Encendido:** los ejes arrancan en una posición al azar. La máquina responde `NOT_HOMED` y rechaza los `MOVE` con `ERR NOT_HOMED`.
2. **Comando `HOME`:** para cada eje, en orden **X → Y → Z**:
   - búsqueda rápida hacia el sensor (25 mm/s) hasta que se activa;
   - se aleja 6 mm;
   - vuelve lento (10 mm/s) y, en el instante en que el sensor se activa, esa posición pasa a ser el **cero** del eje. La pasada lenta da más precisión.
3. Al terminar los tres ejes responde `DONE HOME`: la máquina está en **(0, 0, 0)** y acepta movimientos.

Eje	Sensor de inicio de carrera	Recorrido después del homing
X	Extremo izquierdo	0 a 497 mm
Y	Extremo delantero	0 a 397 mm
Z	Arriba	−177 a 0 mm (negativo = hacia abajo)

Los sensores son los bloques amarillos del modelo. Están 3 mm antes del tope mecánico, como en una máquina real. En la simulación, un sensor se activa cuando el carro llega a su posición; su estado se consulta con `SWITCH?`.

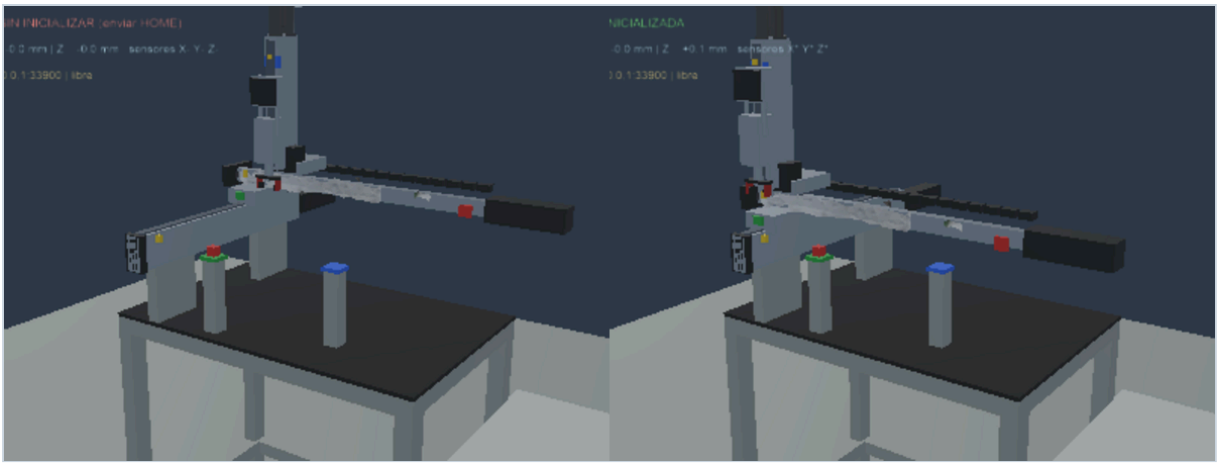


Figura 2. Izquierda: recién encendida, en posición desconocida ("SIN INICIALIZAR"). Derecha: después de `HOME`, en (0, 0, 0) con los tres sensores activados.

4. Protocolo de comunicación

Texto ASCII plano: una línea por mensaje, terminada en `\n`. Distancias en **milímetros** de máquina y velocidades en **mm/s**. La regla es **un comando, una respuesta**: el cliente espera la respuesta antes de mandar el siguiente. Al conectarse, el servidor saluda con `READY CARTESIANA 2.0 NOT_HOMED`.

Comandos inmediatos (se responden aunque la máquina se esté moviendo)

Comando	Respuesta	Uso
`PING`	`PONG`	Comprobar el enlace
`INFO`	`INFO CARTESIANA 2.0 HOMED XMIN 0.0 XMAX 497.1 …`	Estado y recorrido
`POS?`	`POS x y z OPEN|CLOSED HELD|EMPTY HOMED|NOT_HOMED`	Posición, grip e inicialización
`STATUS?`	`STATUS IDLE|BUSY n HOMED|NOT_HOMED`	Ocupada y comandos en cola
`SWITCH?`	`SWITCH X1 Y0 Z0`	Sensores de inicio de carrera
`STOP`	`ERR ABORTED <cmd> … OK STOP`	Frena y descarta lo pendiente

Comandos en cola (se ejecutan de a uno; responden al terminar)

Comando	Respuesta	Uso
`HOME`	`DONE HOME`	Ciclo de inicialización X → Y → Z
`MOVE x y z`	`DONE MOVE x y z`	Mover los tres ejes (requiere HOME)
`SPEED v`	`OK SPEED X40.0 Y40.0 Z25.0`	Velocidad (cada eje la limita a su máximo)
`GRIP CLOSE`	`DONE GRIP CLOSE HELD|EMPTY`	Cerrar el grip; HELD si tomó la pieza
`GRIP OPEN`	`DONE GRIP OPEN [ZOCALO_A|ZOCALO_B|MESA]`	Abrir; indica dónde quedó la pieza
`WAIT ms`	`DONE WAIT`	Pausa de 0 a 60000 ms
`RESET`	`DONE RESET`	Pieza al zócalo A y grip abierto

Errores

Respuesta	Causa
`ERR NOT_HOMED`	`MOVE` antes de inicializar.
`ERR RANGE <eje> <valor> (<min>..<max>)`	Destino fuera del recorrido. La orden se rechaza: **no se mueve**.
`ERR SYNTAX …` / `ERR UNKNOWN …`	Argumentos inválidos o comando inexistente.
`ERR TIMEOUT <cmd>` / `ERR ABORTED <cmd>`	No terminó a tiempo / cortado por `STOP`.
`ERR BUSY` / `ERR TOOLONG`	Ya hay otro cliente / línea de más de 128 caracteres.

Ejemplo real (registro de la prueba)

```
<< READY CARTESIANA 2.0 NOT_HOMED
>> MOVE 100 100 -40     << ERR NOT_HOMED (enviar HOME primero)
>> HOME                 << DONE HOME
>> SWITCH?              << SWITCH X1 Y1 Z1
>> MOVE 100 100 -160    << DONE MOVE 100.0 100.0 -159.6
>> GRIP CLOSE           << DONE GRIP CLOSE HELD
>> MOVE 400 200 -40     << DONE MOVE 399.6 200.0 -40.1
>> MOVE 400 200 -160    << DONE MOVE 400.0 200.0 -159.6
>> GRIP OPEN            << DONE GRIP OPEN ZOCALO_B
```

5. El controlador de Webots (PC industrial simulada)

- **Hereda la cinemática validada** del controlador original (`cartesian_controller`): límites, velocidades y sensores de posición. El mundo original sigue funcionando igual.
- **Servidor TCP no bloqueante:** en cada paso de simulación (16 ms) acepta conexiones y lee lo que llegó, sin congelar Webots.
- **Cola de comandos:** los comandos se ejecutan de a uno. Un movimiento termina cuando los tres ejes quedan a menos de 0.8 mm del objetivo durante 0.2 s.
- **Grip:** dos dedos, cada uno con su `SliderJoint`, `LinearMotor` y `PositionSensor`, colgados del carro Z. Al cerrarse, si la pieza está entre los dedos, el controlador (que es `Supervisor`) la traslada con la punta; al abrirse, la apoya en el zócalo de abajo o en la mesa. Esta simplificación evita depender de la fricción simulada.
- **Zócalos de ejemplo:** no existen en la máquina real; se agregaron para la demostración. Están sobre columnas porque el recorrido Z provisional no llega a la mesa.

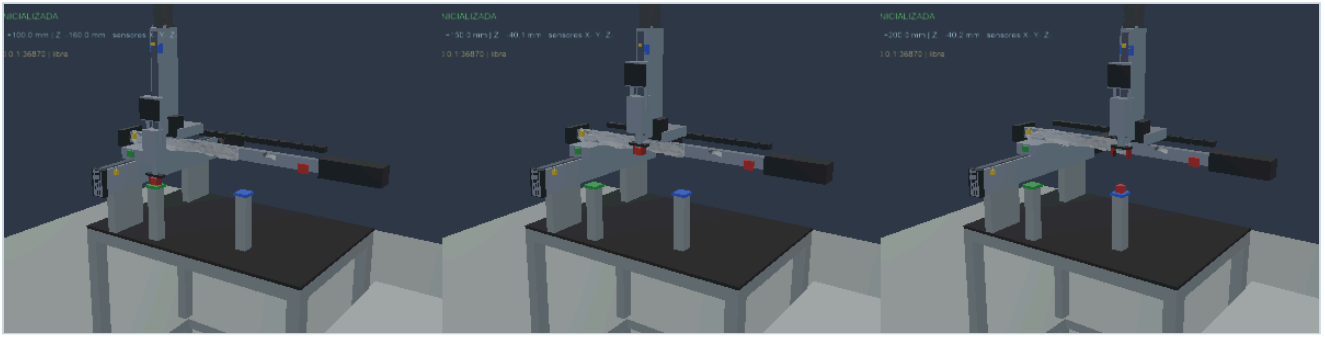


Figura 3. El grip toma la pieza en el zócalo A (verde), la transporta y la deja en el zócalo B (azul).

6. Programa externo

Aplicación de escritorio en Python (Tkinter). Para CARTESIANA no existía, así que se desarrolló como pide el enunciado: registrar puntos de una trayectoria y luego ejecutarla. Funciona como el panel de una máquina industrial.

1. **Conectar** a la IP y el puerto de la máquina (cliente TCP).
2. **Inicializar:** envía `HOME`. Un indicador muestra "SIN INICIALIZAR" (rojo) o "INICIALIZADA" (verde). Sin inicializar no deja mover ni grabar puntos.
3. **Mover a mano** con ±X, ±Y, ±Z o "Ir a X Y Z", y abrir o cerrar el grip.
4. **Guardar punto actual:** agrega la posición a la trayectoria, con una acción de grip y una espera opcionales. Los puntos se editan, reordenan y guardan o cargan como JSON.
5. **Ejecutar trayectoria:** envía `MOVE`, `GRIP` y `WAIT` por cada punto, esperando cada `DONE`. Puede repetirse en ciclo y se corta con **STOP**.

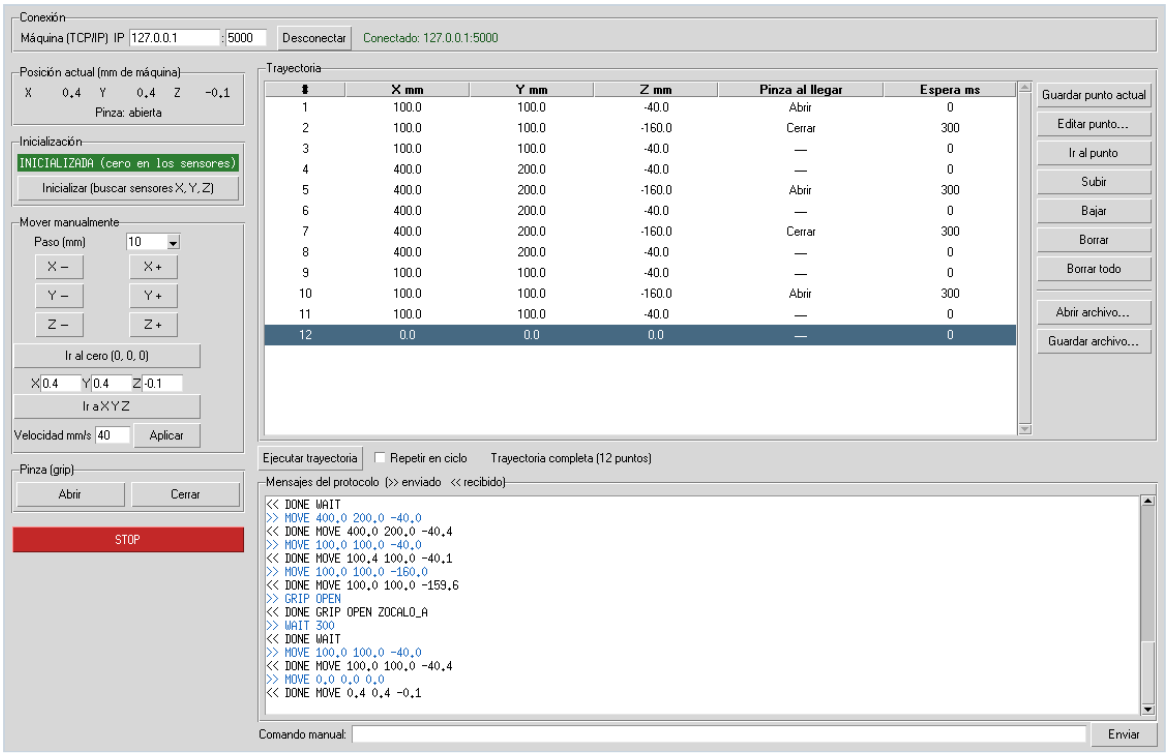


Figura 4. Programa externo conectado por TCP, con la máquina inicializada.

7. Ensayos realizados (Webots R2025a)

Ensayo	Resultado
Arranque en posición desconocida y `MOVE` sin inicializar	`ERR NOT_HOMED`; el grip sí responde.
Ciclo `HOME` desde posiciones al azar (8 corridas)	Siempre `DONE HOME`, con los tres sensores activos y la posición en (0, 0, 0).
Programa externo: conectar → inicializar → `ciclo_A_B_A.json`	Trayectoria completa: la pieza va al zócalo B y vuelve al A.
Límites, sintaxis, `STOP`, segundo cliente	`ERR RANGE`, `ERR SYNTAX`, `ERR ABORTED` + `OK STOP`, `ERR BUSY`.
Validación cinemática original con el modelo nuevo	23/23 comprobaciones aprobadas.

8. Limitaciones

- Recorridos, masas y medidas del grip son provisionales: no se midieron en la máquina real. Por eso los zócalos de ejemplo están elevados.
- La posición de los sensores se supuso (X a la izquierda, Y adelante, Z arriba). Si en la máquina están en otros extremos, se cambia en `config/robot_parameters.json` (`homing.switch_at`).
- El agarre lo resuelve el Supervisor, no la fricción. Si la pieza no está entre los dedos al cerrar, la respuesta es `EMPTY`.
- Un solo cliente a la vez.